React 使用的 ES6 語法

什麼是 ES6？
ES6 代表 ECMAScript 6。

ECMAScript 是為了規範 JavaScript 而建立的，ES6 是 ECMAScript 的第 6 個版本，於 2015 年發布，也被稱為 ECMAScript 2015。

1. React ES6 Classes
   ES6 引入了類別。
   類別是一種函數，但我們不使用關鍵字 function 來啟動它，而是使用關鍵字 class，並且在方法內部分配屬性 constructor()。

```
class Car {
  constructor(name) {
    this.brand = name;
  }
}
```

範例:

```
<!DOCTYPE html>
<html>
<body>
<script>
class Car {
  constructor(name) {
    this.brand = name;
  }
}
const mycar = new Car("Ford");
document.write(mycar.brand);
</script>

</body>
</html>
```

類別內定義方法:

```
<!DOCTYPE html>
<html>
<body>
<script>
class Car {
  constructor(name) {
    this.brand = name;
  }

  present() {
    return 'I have a ' + this.brand;
  }
}
const mycar = new Car("Ford");
document.write(mycar.present());
</script>

</body>
</html>
```

類別繼承

要建立類別繼承，請使用 extends 關鍵字。
使用類別繼承所建立的類別可以繼承另一個類別的所有方法
super()方法引用父類別。
通過調用 super()建構方法，我們調用了父類別的建構方法，並獲得了對父類別的屬性和方法的訪問權。

```
<!DOCTYPE html>
<html>
<body>
<script>
class Car {
  constructor(name) {
    this.brand = name;
  }
```

```
    present() {
        return 'I have a ' + this.brand;
    }
}

class Model extends Car {
    constructor(name, mod) {
        super(name);
        this.model = mod;
    }
    show() {
        return this.present() + ', it is a ' + this.model
    }
}

const mycar = new Model("Ford", "Mustang");
document.write(mycar.show());
</script>
</body>
</html>
```

2. React ES6 Arrow Functions

先前:

```
hello = function() {
    return "Hello World!";
}
```

目前 Arrow Function:

```
hello = () => {
    return "Hello World!";
}
```

範例:

```
<!DOCTYPE html>
<html>

<body>

<h1>Arrow Function</h1>
<p>The <strong>this</strong> keyword represents the Header object.</p>
<button id="btn">Click Me!</button>
<p><strong>this</strong> represents:</p>

<p id="demo"></p>
<script>
class Header {
  constructor() {
    this.color = "Red";
  }

  changeColor = () => {
    document.getElementById("demo").innerHTML += this.color;
  }
}

const myheader = new Header();

//The window object calls the function:
window.addEventListener("load", myheader.changeColor);

//A button object calls the function:
document.getElementById("btn").addEventListener("click", myheader.changeColor);

</script>
</body>
</html>
```

3. Array Methods---map()

React 中最有用的方法之一是.map()陣列方法。

該.map()方法允許您對陣列中的每個元素執行一個函數，返回一個新陣列作為結果

```
<!DOCTYPE html>
<html>
<head>
	<meta charset="utf-8" />
	<title>Array Map</title>
</head>
<script>
	const myArray = ['apple', 'banana', 'orange'];
	const myList = myArray.map((item) => "<p>" + item + "</p>");
	alert(myList);
</script>
</body>
</html>
```

```
import React from 'react';

const App = () => {
  const numbers = [1, 2, 3, 4, 5];

  // 使用 map() 方法將每個數字乘以 2
  const doubledNumbers = numbers.map((number) => number * 2);

  // 使用 filter() 方法過濾出大於 3 的數字
  const filteredNumbers = numbers.filter((number) => number > 3);

  // 使用 reduce() 方法計算所有數字的總和
  const sum = numbers.reduce((accumulator, currentNumber) => accumulator + currentNumber, 0);

  return (
    <div>
      <h1>React App</h1>
      <h2>Doubled Numbers: {doubledNumbers.join(', ')}</h2>
      <h2>Filtered Numbers: {filteredNumbers.join(', ')}</h2>
```

```
            <h2>Sum: {sum}</h2>
        </div>
    );
};

export default App;
```

4. ES6 Destructuring

Destructuring 是將正在使用的陣列或物件，將我們需要其中的一些欄位取出。

舊的語法:

```
const vehicles = ['mustang', 'f-150', 'expedition'];

// old way
const car = vehicles[0];
const truck = vehicles[1];
const suv = vehicles[2];
```

使用 destructuring:

```
const vehicles = ['mustang', 'f-150', 'expedition'];
const [car, truck, suv] = vehicles;
```

範例:

```
<!DOCTYPE html>
<html>
<body>
<script>
function calculate(a, b) {
    const add = a + b;
    const subtract = a - b;
    const multiply = a * b;
    const divide = a / b;

    return [add, subtract, multiply, divide];
}
const [add, subtract, multiply, divide] = calculate(4, 7);

document.write("<p>Sum: " + add + "</p>");
document.write("<p>Difference " + subtract + "</p>");
document.write("<p>Product: " + multiply + "</p>");
document.write("<p>Quotient " + divide + "</p>");
</script>

</body>
</html>
```

Destructuring Objects

舊物件語法:

```
const vehicleOne = {
    brand: 'Ford',
    model: 'Mustang',
    type: 'car',
    year: 2021,
    color: 'red'
}

myVehicle(vehicleOne);

// old way
```

```
function myVehicle(vehicle) {
    const message = 'My ' + vehicle.type + ' is a ' + vehicle.color + ' ' + vehicle.brand + '
' + vehicle.model + '.';
}
```

ES6 destructuring 語法:

```
const vehicleOne = {
    brand: 'Ford',
    model: 'Mustang',
    type: 'car',
    year: 2021,
    color: 'red'
}

myVehicle(vehicleOne);

function myVehicle({type, color, brand, model}) {
    const message = 'My ' + type + ' is a ' + color + ' ' + brand + ' ' + model + '.';
}
```

範例:

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>

<script>
const vehicleOne = {
    brand: 'Ford',
    model: 'Mustang',
    type: 'car',
    year: 2021,
    color: 'red'
}

myVehicle(vehicleOne);
```

```
function myVehicle({type, color, brand, model}) {
   const message = 'My ' + type + ' is a ' + color + ' ' + brand + ' ' + model + '.';

   document.getElementById("demo").innerHTML = message;
}
</script>

</body>
</html>
```

範例 2:

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script>
const vehicleOne = {
   brand: 'Ford',
   model: 'Mustang',
   type: 'car',
   year: 2021,
   color: 'red',
   registration: {
      city: 'Houston',
      state: 'Texas',
      country: 'USA'
   }
}
myVehicle(vehicleOne);
function myVehicle({ model, registration: { state } }) {
   const message = 'My ' + model + ' is registered in ' + state + '.';

   document.getElementById("demo").innerHTML = message;
}
</script>
</body>
</html>
```

5. Spread Operator

JavaScript 擴充運算符號（...）允許我們快速將現有陣列或物件的全部或部分複製到另一個陣列或物件中。

例子

```
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];
```

```
<!DOCTYPE html>
<html>
<body>
<script>
const numbers = [1, 2, 3, 4, 5, 6];
const [one, two, ...rest] = numbers;
document.write("<p>" + one + "</p>");
document.write("<p>" + two + "</p>");
document.write("<p>" + rest + "</p>");
</script>
</body>
</html>
```

結果:

1

2

3,4,5,6

範例:

```
<!DOCTYPE html>
<html>
<body>
<script>
const myVehicle = {
    brand: 'Ford',
    model: 'Mustang',
    color: 'red'
```

```
}

const updateMyVehicle = {
   type: 'car',
   year: 2021,
   color: 'yellow'
}

const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle}

//Check the result object in the console:
console.log(myUpdatedVehicle);
</script>

<p>Press F12 and see the result object in the console view.</p>

</body>
</html>
```

結果:
{brand: 'Ford', model: 'Mustang', color: 'yellow', type: 'car', year: 2021}

6. React ES6 Modules

JavaScript Module 允許您將程式碼分解成單獨的文件。
這使得維護程式碼庫變得更加容易。
ES6 模塊依賴於 import 及 export 語句。

export
您可以從任何文件導出函式或變數。
讓我們建立一個名為 的 person.js 文件，並在其中填充我們要導出的內容。

有兩種類型的導出：具名和默認。

默認導出(default export)

讓我們建立一個名為，message.js 的文件並將其用於展示默認導出。
一個文件中只能有一個默認導出。

例子
message.js

```
const message = () => {
    const name = "Jesse";
    const age = 40;
    return name + ' is ' + age + 'years old.';
};
export default message;
```

person.js

```
const name = "Jesse"
const age = 40

export { name, age }
```

import

您可以通過兩種方式將模塊導入到文件中，具體取決於它們是具名為 exports 還是 default exports。

具名導出必須使用大括號進行解構。
從文件 person.js 導入具名導出：

```
import { name, age } from "./person.js";
```

範例:

```
const message = () => {
	const name = "Jesse";
	const age = 40;
	return name + ' is ' + age + ' years old.';
};
const name = "Jesse"
const age = 40

export default message;
export { name, age }
```

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script type="module">
import { name, age } from "./person.js";
document.getElementById("demo").innerHTML = "My name is " + name;
document.getElementById("demo").innerHTML += ", I am " + age + ".";
</script>
</body>
</html>
```

範例:

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script type="module">
import message from "./message.js";
document.getElementById("demo").innerHTML = message();

</script>
</body>
</html>
```